

TEMA 6 - CSS GRID Y LAYOUTS COMPLEJOS

Bidimensional

Qué es CSS Grid

- Sistema de **dos dimensiones**: filas + columnas
- Complementa a Flexbox (Flex → 1 dimensión)
- Ideal para **layouts completos de página**

Contenedor Grid

```
.contenedor {  
  display: grid;  
  
  grid-template-columns: 100px 200px 100px;  
  
  grid-template-rows: 50px 100px;  
  
  gap: 10px;  
}
```

- `grid-template-columns` → define columnas
- `grid-template-rows` → define filas
- `gap` → espacio entre celdas

Ejercicio 1

HTML base:

```
<div class="contenedor">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
</div>
```

Objetivo:

- Definir **2 filas** y **2 columnas**
- Separar elementos con **gap: 15px**
- Observar cómo se coloca cada ítem automáticamente

Unidades fr

fr → fracción del espacio disponible

```
grid-template-columns: 1fr 2fr 1fr;
```

- Total: 4 fr
- Segunda columna → ocupa **la mitad del contenedor**

Ejercicio 2

Layout proporcional

- Crear un grid con 3 columnas
- Asignar anchos: `2fr 1fr 1fr`
- Observar cómo la primera columna ocupa el doble que las demás

Filas automáticas

```
grid-auto-rows: 100px;
```

- Define altura por defecto para filas que se generen automáticamente

Ejercicio 3

Grid con 4 items, 2 filas automáticas

Asignar `grid-auto-rows: 80px`

Observar cómo se adaptan las filas según el contenido

Grid areas

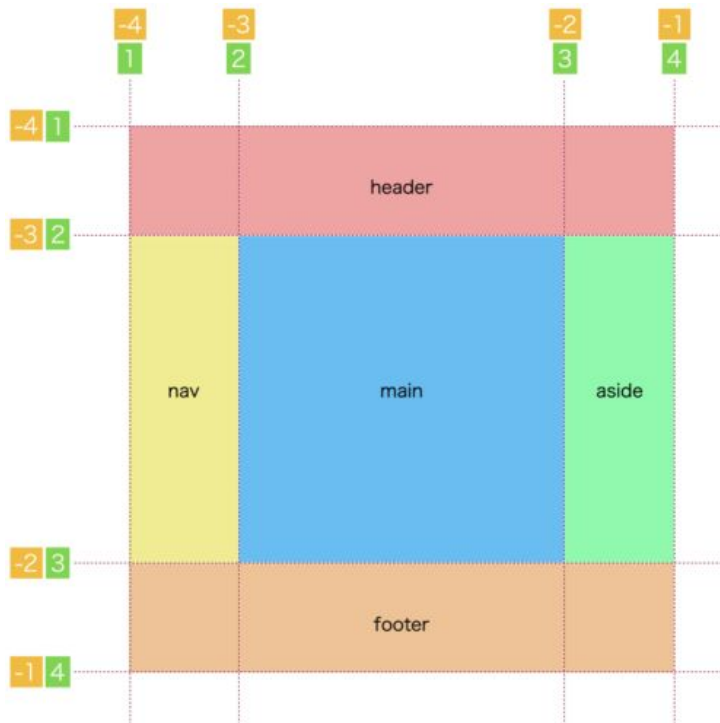
`grid-template-areas:`

`"header header header"`

`"menu main sidebar"`

`"footer footer footer";`

- Permite **nombrar áreas**
- Facilita **layouts tipo web real**



Layout

Objetivo:

- Crear layout clásico de página: Header + Menu + Main + Sidebar + Footer
- Usar `grid-template-areas`
- Usar `gap` para separar celdas
- Observar cómo se colocan los elementos en su área

HTML base:

```
<div class="grid-contenedor">  
  <header>Header</header>  
  <nav>Menu</nav>  
  <main>Main</main>  
  <aside>Sidebar</aside>  
  <footer>Footer</footer>  
</div>
```

Posicionamiento de items

Propiedades importantes:

```
grid-column: 2 / 4; /* desde columna 2 hasta 3 */
```

```
grid-row: 1 / 3;    /* desde fila 1 hasta 2 */
```

Permite **superponer o extender items** en varias filas o columnas.

Ejercicio 5

Requisitos:

- Crear un **dashboard de 3 columnas x 3 filas**
- Header → ocupa fila completa superior
- Sidebar → columna 1, filas 2-3
- Content → columna 2-3, fila 2
- Ads → columna 2-3, fila 3
- Footer → fila inferior completa

```
<div class="dashboard">  
  <div class="item  
header">Header</div>  
  <div class="item  
sidebar">Sidebar</div>  
  <div class="item  
content">Content</div>  
    <div class="item ads">Ads</div>  
  <div class="item  
footer">Footer</div>  
</div>
```

Alineación dentro de Grid

- `justify-items` → eje horizontal
- `align-items` → eje vertical
- `justify-content` → todo el grid horizontal
- `align-content` → todo el grid vertical

```
.item {  
  justify-self: center;  
  align-self: start;  
}
```

Ejercicio 6

- Crear un grid de 3x2
- Cada carta (grid-item) centrada vertical y horizontalmente dentro de su celda
- Usar `justify-self` y `align-self`

Unidades útiles

`%, px, fr, auto`

`minmax(100px, 1fr)` → tamaño mínimo / flexible

`repeat(3, 1fr)` → repetir columnas

Ejercicio 7

Haz un grid responsive con:

- 4 items
- Columnas: `repeat(auto-fit, minmax(150px, 1fr))`
- Observa cómo se adaptan al ancho de pantalla
- Ejercicio útil para maquetar tarjetas de productos

Buenas prácticas

- Combinar Flexbox para elementos internos (1 dimensión)
- Usar Grid para **layouts generales**
- Separar areas con **gap** en lugar de margin
- Evitar floats para layout complejo